

CSC3060 Project 3: Code Scheduling

1. Problem (Instruction abstraction [5 point]):

See the codes `useReg()` and `definesReg()` in `src/Instruction.cpp`

2. Problem (Static latency model [2+2+5 point]):

- (a) Load must wait for data to return from memory (or cache) before the result can be used by subsequent instructions, so it has high latency.
Store only needs to write data into a write buffer and can return immediately, it does not produce a register result that later instructions depend on, so it effectively takes 1 cycle from the scheduler's perspective.
- (b) A static latency model provides a reasonable compile-time approximation that works well for the common case. The compiler cannot predict runtime conditions like cache misses, so using fixed latencies based on the most likely scenario still produces better schedules than no scheduling at all.
- (c) See the code `getInstructionLatency()` in `src/Instruction.cpp`

3. Problem (Dependency Detection [2+2+10+10+[5] point]):

- (a) Two instructions that both read the same register do not conflict with each other because reading does not modify the register's value. Since neither instruction changes the shared state, they can execute in any order without affecting correctness.
- (b) A dependency graph must be a DAG because instructions in a basic block have a linear program order, and dependencies only point forward (from earlier to later instructions), so cycles cannot exist. If cycles existed, it would mean a set of instructions mutually depend on each other or indirectly depend on themselves, making it impossible to find a valid execution order and any valid topological order for scheduling.
- (c) See the codes: `detectWAR()`, `detectWAW()` (`src/DAGBuilder.cpp`)
- (d) Bonus: The core idea: originally, each of the three passes maintained its own map, but two maps can be shared and processed simultaneously in a single traversal.
See the code `DAGBuilder::build`

4. Problem (Priority Calculation [4+10+10 point]):

- (a) WAR only requires that the earlier instruction has read the register before the later one overwrites it, and WAW only requires that the two writes occur in the correct order — both are guaranteed at issue time without waiting for execution to complete. But RAW carries a true timing constraint, because the successor must wait for the produced value to

become available.

- (b) See the code: `computeCriticalPath()` (`src/Scheduler.cpp`)
- (c) See the code: `schedule()` (`src/Scheduler.cpp`)

5. Problem (Tie-Breaking Policies [2+2+10 point]):

- (a) Scheduling a node with more successors first can unblock a larger number of dependent instructions sooner, increasing the pool of ready instructions in future cycles and giving the scheduler more flexibility to hide latencies.
For example, if node A has 3 successors and node B has 1, issuing A first potentially makes 3 new instructions available rather than just 1.
- (b) When two ready instructions have the same priority but different latencies, LPT starts the long-latency instruction earlier, allowing shorter independent instructions to execute during its long execution window and hide the latency.
For example, if a 3-cycle load and a 1-cycle add are both ready with equal priority, issuing the load first lets the add fill one of the load's stall slots.
- (c) See the code: `selectBestNode()` (`src/Scheduler.cpp`)

6. Problem (Complexity Analysis [2+2+[5] point]):

- (a) $O(n^2 + e)$, where n is the number of instructions and e is the number of dependency edges. The main loop runs $O(n)$ iterations (at most one instruction is scheduled per iteration). Each iteration calls `getReadyNodes()`, which scans all n nodes in $O(n)$ time, contributing $O(n^2)$ total. Across the entire scheduling process, each dependency edge is examined at most a constant number of times during successor updates, contributing $O(e)$. The dominant term is $O(n^2)$ from the repeated node scanning.
- (b) $O(n + e)$, where n is the number of instructions and e is the number of dependency edges. Each node is visited exactly once due to the visited array (contributing $O(n)$), and each edge is examined exactly once when processing its source node's successors (contributing $O(e)$). This is essentially a single DFS traversal of the DAG.
- (d) Bonus: See the code `scheduleFast()` (`src/Scheduler.cpp`)

7. Problem (Run Scheduler [10+10 point]):

- (a) See the code `runScheduler()` (`src/main.cpp`)

Example 1 (`getExampleCode()`): Expression Calculation): Scheduling does not improve performance for this example (Original order total cycles: 16 cycles; Scheduled order total cycles: 16 cycles). The critical path (`lw` $\rightarrow$ `mul` $\rightarrow$ `add` $\rightarrow$ `mul` $\rightarrow$ `sw`) determines the minimum execution time, and no reordering can shorten it. The scheduler

moves the independent `lw x14` earlier to fill an idle slot, but this does not reduce total cycles since the bottleneck is the data dependency chain, not idle slots (See Figure 1, 2, 3).

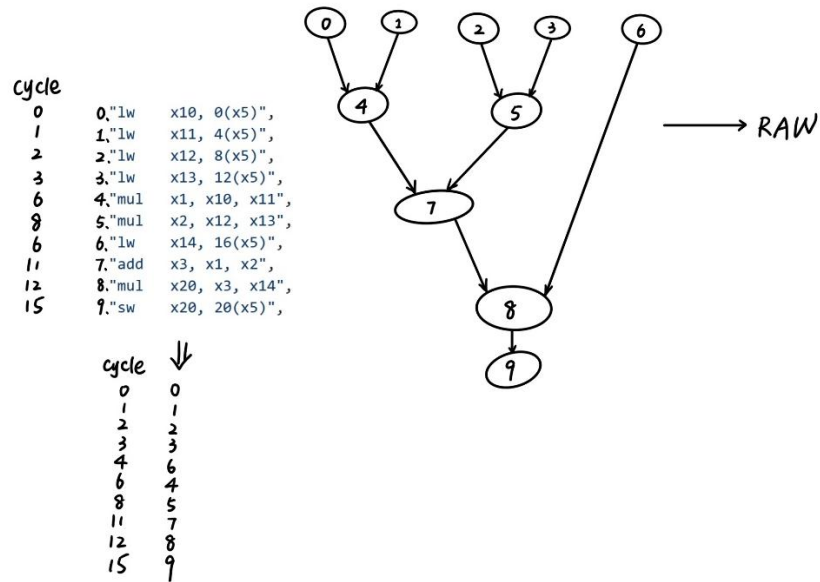


Figure 1: DAG and Analysis of Example 1

Example 1: Expression Calculation		
Scheduling Result		
Original instruction order:		
0:	lw	x10, 0(x5)
1:	lw	x11, 4(x5)
2:	lw	x12, 8(x5)
3:	lw	x13, 12(x5)
4:	mul	x1, x10, x11
5:	mul	x2, x12, x13
6:	lw	x14, 16(x5)
7:	add	x3, x1, x2
8:	mul	x20, x3, x14
9:	sw	x20, 20(x5)
Scheduled instruction order:		
0:	[Orig 0]	lw x10, 0(x5) (priority: 13)
1:	[Orig 1]	lw x11, 4(x5) (priority: 13)
2:	[Orig 2]	lw x12, 8(x5) (priority: 13)
3:	[Orig 3]	lw x13, 12(x5) (priority: 13)
4:	[Orig 6]	lw x14, 16(x5) (priority: 9)
5:	[Orig 4]	mul x1, x10, x11 (priority: 8)
6:	[Orig 5]	mul x2, x12, x13 (priority: 8)
7:	[Orig 7]	add x3, x1, x2 (priority: 5)
8:	[Orig 8]	mul x20, x3, x14 (priority: 4)
9:	[Orig 9]	sw x20, 20(x5) (priority: 1)
Scheduling statistics:		
Instruction count: 10		

Figure 2: Scheduling Result Comparison of Example 1

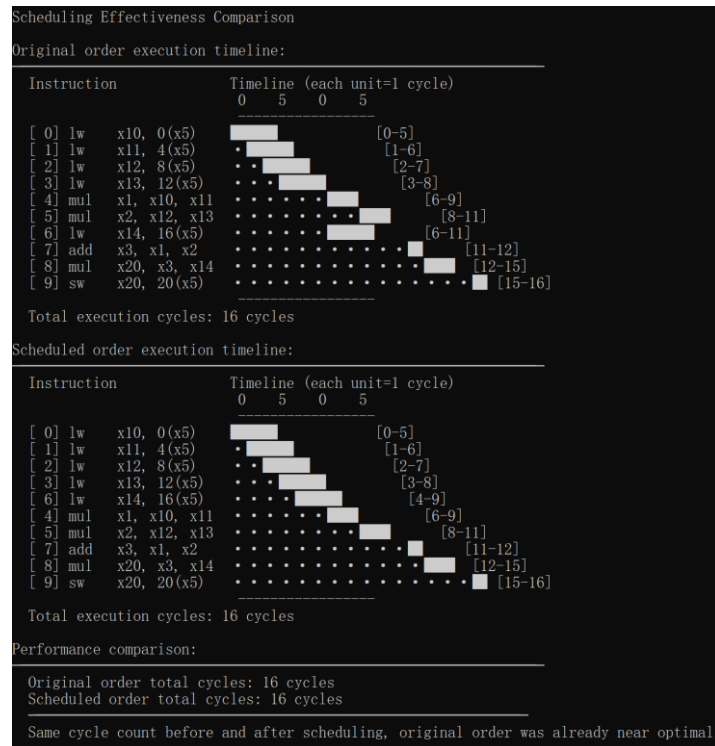


Figure 3: Scheduling Effectiveness Comparison of Example 1

Example 2 (getExampleCode2(): Dependency Type Demonstration): Scheduling provides limited or no improvement (Original order total cycles: 7 cycles; Scheduled order total cycles: 7 cycles). Register x1 is written, read, overwritten, and read again, creating RAW, WAR, and WAW dependencies that lock every instruction to its original position. The scheduler has zero reordering freedom. The instructions form a tight dependency chain with little opportunity to overlap independent work (See Figure 4, 5, 6).

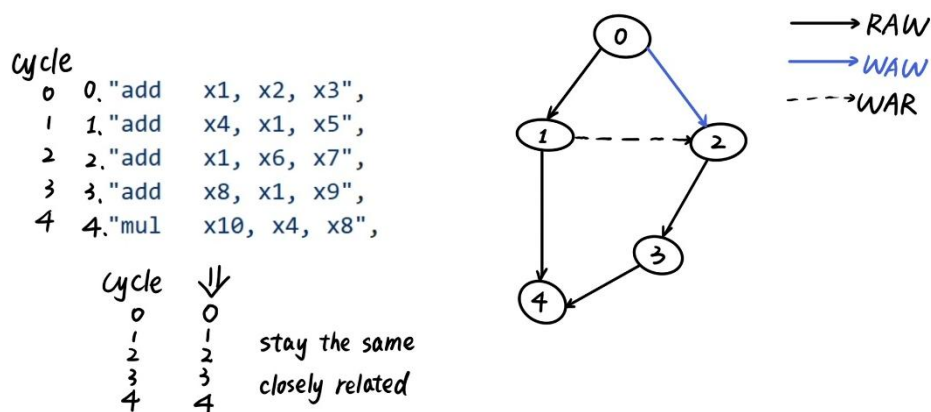


Figure 4: DAG and Analysis of Example 2

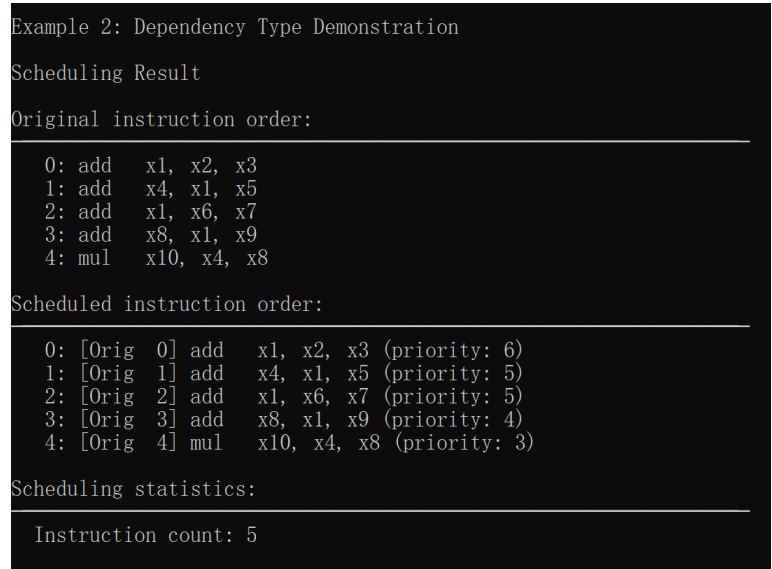


Figure 5: Scheduling Result Comparison of Example 2

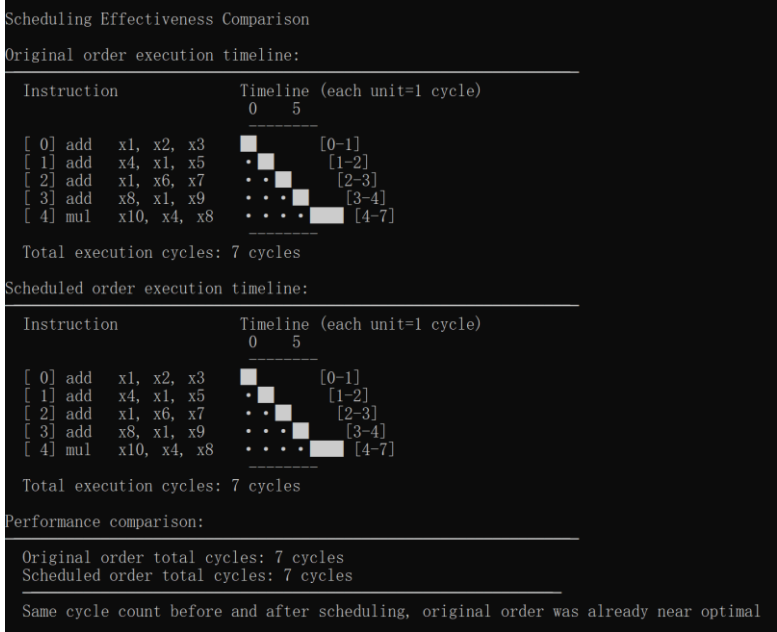


Figure 6: Scheduling Effectiveness Comparison of Example 2

Example 3 (getExampleCode3_BadOrder()): Load-Use Hazard Hidden):

Scheduling significantly improves performance (Original order total cycles: 17 cycles; Scheduled order total cycles: 14 cycles; Cycles saved: 3 cycles; Performance improvement: 17.6%). The original code interleaves each `lw` with its dependent `mul` immediately after, creating a load-use hazard every time. The scheduler groups all `lw` instructions together at the beginning, so by the time the first `mul` is issued, its source data is already available. This effectively hides all load latencies (See Figure 7, 8, 9).

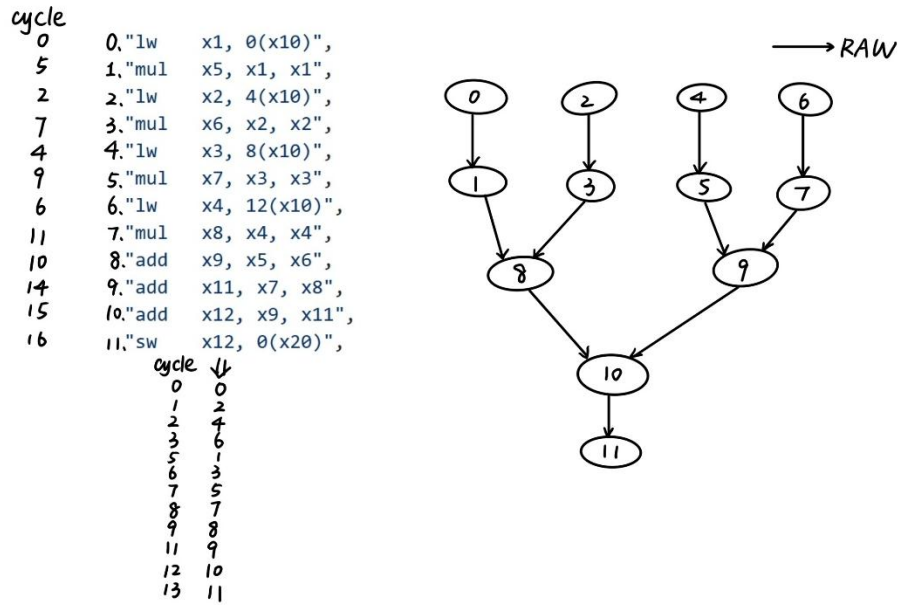


Figure 7: DAG and Analysis of Example 3

Example 3: Load-Use Hazard Hidden

Scheduling Result

Original instruction order:

```

0: lw    x1, 0(x10)
1: mul   x5, x1, x1
2: lw    x2, 4(x10)
3: mul   x6, x2, x2
4: lw    x3, 8(x10)
5: mul   x7, x3, x3
6: lw    x4, 12(x10)
7: mul   x8, x4, x4
8: add   x9, x5, x6
9: add   x11, x7, x8
10: add  x12, x9, x11
11: sw   x12, 0(x20)

```

Scheduled instruction order:

```

0: [Orig 0] lw    x1, 0(x10) (priority: 11)
1: [Orig 2] lw    x2, 4(x10) (priority: 11)
2: [Orig 4] lw    x3, 8(x10) (priority: 11)
3: [Orig 6] lw    x4, 12(x10) (priority: 11)
4: [Orig 1] mul   x5, x1, x1 (priority: 6)
5: [Orig 3] mul   x6, x2, x2 (priority: 6)
6: [Orig 5] mul   x7, x3, x3 (priority: 6)
7: [Orig 7] mul   x8, x4, x4 (priority: 6)
8: [Orig 8] add   x9, x5, x6 (priority: 3)
9: [Orig 9] add   x11, x7, x8 (priority: 3)
10: [Orig 10] add  x12, x9, x11 (priority: 2)
11: [Orig 11] sw   x12, 0(x20) (priority: 1)

```

Scheduling statistics:

Instruction count: 12

Figure 8: Scheduling Result Comparison of Example 3

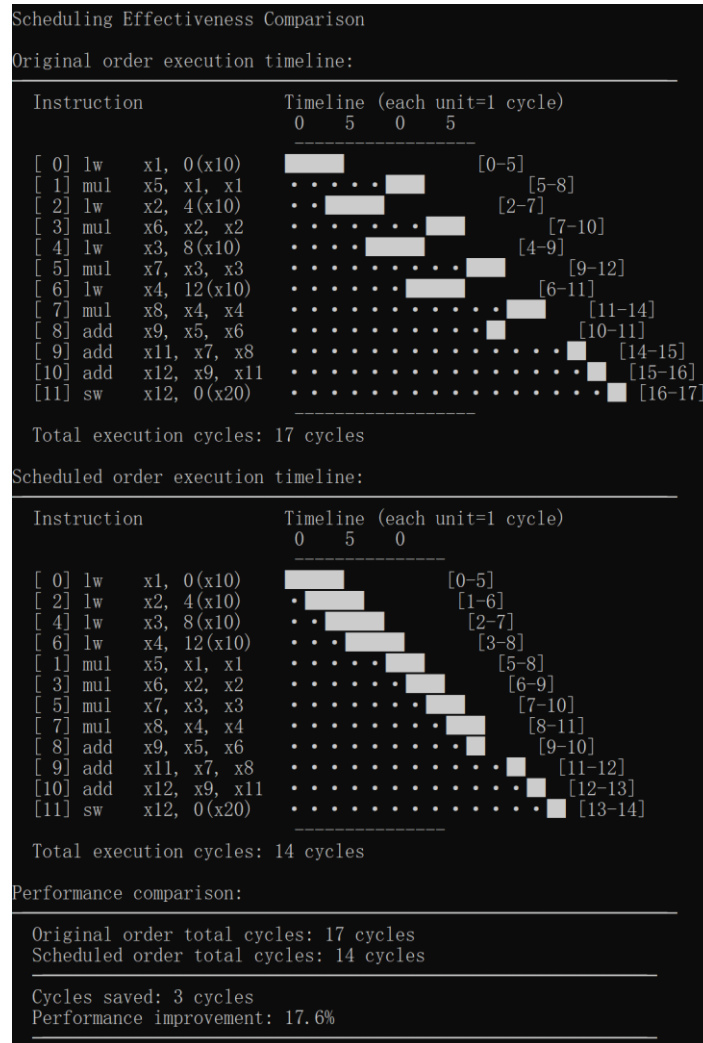


Figure 9: Scheduling Effectiveness Comparison of Example 3

Example 4 (getExampleCode4_CriticalPath(): Early Start Critical Operation):

Scheduling improves performance by identifying the critical path through the `div` instruction (10-cycle latency) and starting it as early as possible (Original order total cycles: 21 cycles; Scheduled order total cycles: 16 cycles; Cycles saved: 5 cycles; Performance improvement: 23.8%). The `div` has the highest priority due to its long latency chain, so the scheduler issues it early and fills its execution window with independent `add` and `mul` instructions that would otherwise be idle (See Figure 10, 11, 12).

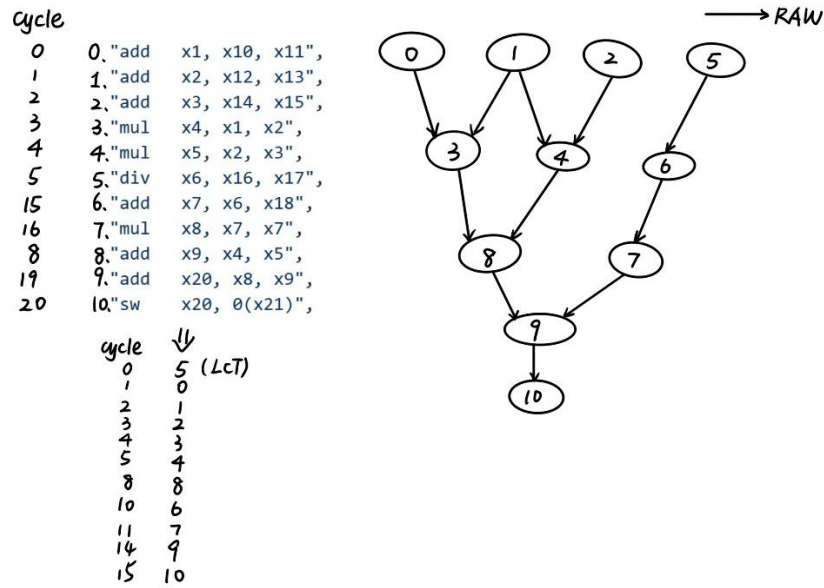


Figure 10: DAG and Analysis of Example 4

Example 4: Early Start Critical Operation

Scheduling Result

Original instruction order:

0:	add	x1, x10, x11
1:	add	x2, x12, x13
2:	add	x3, x14, x15
3:	mul	x4, x1, x2
4:	mul	x5, x2, x3
5:	div	x6, x16, x17
6:	add	x7, x6, x18
7:	mul	x8, x7, x7
8:	add	x9, x4, x5
9:	add	x20, x8, x9
10:	sw	x20, 0(x21)

Scheduled instruction order:

0:	[Orig 5]	div	x6, x16, x17	(priority: 16)
1:	[Orig 0]	add	x1, x10, x11	(priority: 7)
2:	[Orig 1]	add	x2, x12, x13	(priority: 7)
3:	[Orig 2]	add	x3, x14, x15	(priority: 7)
4:	[Orig 3]	mul	x4, x1, x2	(priority: 6)
5:	[Orig 4]	mul	x5, x2, x3	(priority: 6)
6:	[Orig 8]	add	x9, x4, x5	(priority: 3)
7:	[Orig 6]	add	x7, x6, x18	(priority: 6)
8:	[Orig 7]	mul	x8, x7, x7	(priority: 5)
9:	[Orig 9]	add	x20, x8, x9	(priority: 2)
10:	[Orig 10]	sw	x20, 0(x21)	(priority: 1)

Scheduling statistics:

Instruction count: 11

Figure 11: Scheduling Result Comparison of Example 4

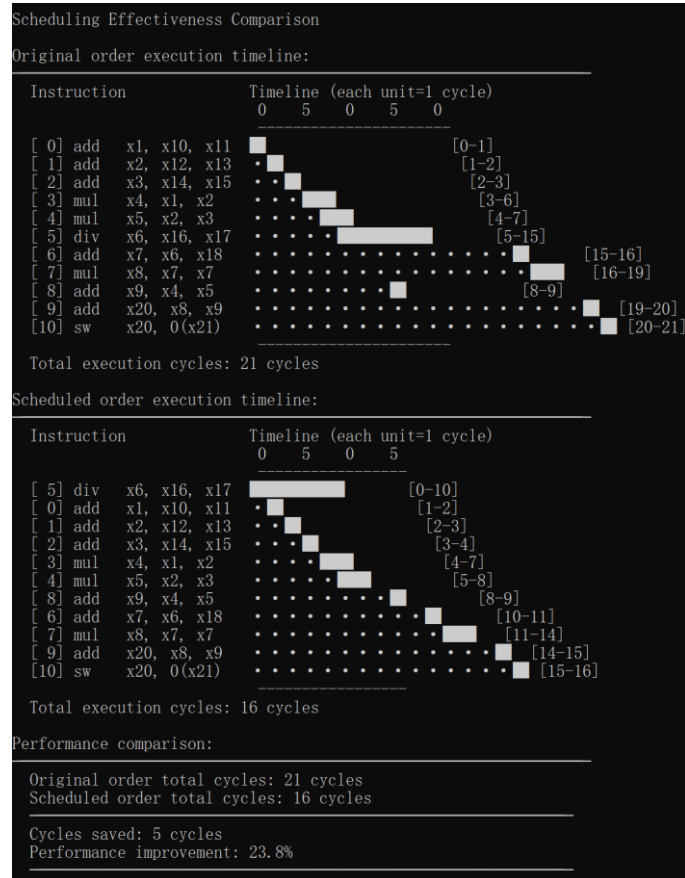


Figure 12: Scheduling Effectiveness Comparison of Example 4

General finding: List scheduling is most effective when there are long-latency instructions (loads, multiplications, divisions) with independent instructions available to fill the latency gaps (Example 3, 4). It provides no benefit when dependency chains are too tight (Example 2) or when the critical path already dominates (Example 1).

AI Use

- 1) I utilized AI to complete this report, including translating Chinese content into English, polishing the wording of sentences, and revising the overall structure.
- 2) I used AI tools to assist in understanding the technical content in the report. For instance, I asked AI to explain the pseudocode, such as the one presented in “Algorithm 1 Detect Read-After-Write (RAW) Dependencies”. Below is an example of a complete dialogue:

```
Algorithm 1 Detect Read-After-Write (RAW) Dependencies
Require: Instruction sequence  $I = (I_0, I_1, \dots, I_{n-1})$ 
Require: DAG node list  $N$ 
Ensure: RAW dependency edges added to  $N$ 
1: Initialize empty map  $LastWriter$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $inst \leftarrow I_i$ 
4:   for all  $r \in inst.src\_regs$  do
5:     if  $r \in LastWriter$  then
6:        $j \leftarrow LastWriter[r]$ 
7:        $lat \leftarrow I_j.latency$ 
8:       Add edge  $(j \rightarrow i)$  of type RAW on register  $r$ 
9:     end if
10:  end for
11:  if  $inst$  writes a destination register  $d$  then
12:     $LastWriter[d] \leftarrow i$ 
13:  end if
14: end for
```

我在根据这个伪代码完成代码，请你给我分析一下这个伪代码的步骤，我主要想关注这里的LastWriter是干什么的

已完成思考 >

🔥 算法整体功能

这个伪代码是**检测指令序列中的 Read-After-Write (RAW) 数据依赖**，并将所有 RAW 依赖以边的形式添加到指令依赖图 (DAG) 的节点列表 N 中。

RAW 依赖的核心是：**后一条指令读取了前一条指令写入的寄存器**，因此两条指令必须按顺序执行。

🔍 伪代码逐步解析 (重点关注 LastWriter)

1. 初始化阶段

```
plaintext ^
1: Initialize empty map LastWriter
```

- LastWriter 是一个**键值对映射 (字典)**：
 - 键：寄存器名称 (如 r1, r2)
 - 值：**最后写入该寄存器的指令索引**
- 作用：追踪每个寄存器的「最新写入者」，为后续检测 RAW 依赖提供快速查询。

2. 遍历指令序列（外层循环）

plaintext ^

📄 ↩ 🔍 ↲

```
2: for i ← 0 to n - 1 do
3:   inst ← I_i
```

- 按顺序遍历指令序列 I 中的每一条指令 I_i ，索引 i 是当前指令的编号。
- 将当前指令赋值给 `inst`，方便后续操作。

3. 检测 RAW 依赖（内层循环：处理读操作）

plaintext ^

📄 ↩ 🔍 ↲

```
4:   for all r ∈ inst.src_regs do
5:     if r ∈ LastWriter then
6:       j ← LastWriter[r]
7:       lat ← I_j.latency
8:       Add edge (j → i) of type RAW on register r
9:     end if
10:  end for
```

这是 `LastWriter` 最核心的使用场景：

- 遍历源寄存器**：当前指令 `inst` 要读取的所有寄存器（`src_regs`）。
- 查询写入历史**：如果寄存器 r 存在于 `LastWriter` 中，说明之前有指令写入过 r ，存在 RAW 依赖。
- 定位写入指令**：`j = LastWriter[r]` 直接拿到「最后写入 r 的指令索引」。
- 添加依赖边**：在 DAG 中添加一条从 j 指向 i 的边，标记为 **RAW 类型**，表示指令 I_i 必须等待 I_j 完成写入后才能读取寄存器 r 。

✅ 这里 `LastWriter` 的作用：**快速找到「谁写了我要读的寄存器」**，避免遍历所有历史指令，让依赖检测效率达到 $O(1)$ 。

4. 更新写入记录（处理写操作）

plaintext ^

📄 ↩ 🔍 ↲

```
11:   if inst writes a destination register d then
12:     LastWriter[d] ← i
13:   end if
14: end for
```

- 当当前指令 `inst` 会写入一个目的寄存器 d 时：
 - 把 `LastWriter[d]` 更新为当前指令索引 i ，覆盖旧值。
 - 作用：**保证 `LastWriter` 永远记录「寄存器的最新写入者」**，让后续指令读 d 时，能检测到正确的 RAW 依赖。

✅ 这里 `LastWriter` 的作用：**更新「最新写入者」**，确保后续依赖检测的准确性，不会遗漏新的写操作。

LastWriter 核心作用总结

表格

阶段	操作	LastWriter 的作用
读操作	查询 LastWriter[r]	快速找到「最后写入寄存器 r 的指令」，检测并建立 RAW 依赖边
写操作	更新 LastWriter[d]	记录「当前指令是寄存器 d 的最新写入者」，为后续读操作的依赖检测提供准确依据

一句话概括：

LastWriter 是寄存器的「写入备忘录」，专门用来追踪「每个寄存器最后被谁写了」，是实现高效、准确检测 RAW 依赖的核心工具。

💡 极简示例（帮你直观理解）

假设指令序列：

- 1. 指令 0：写寄存器 r1 → LastWriter[r1] = 0
- 2. 指令 1：读寄存器 r1 → 查 LastWriter[r1] = 0 → 建立 RAW 边 0→1
- 3. 指令 2：写寄存器 r1 → LastWriter[r1] = 2（覆盖旧值）
- 4. 指令 3：读寄存器 r1 → 查 LastWriter[r1] = 2 → 建立 RAW 边 2→3

可以看到，LastWriter 始终记录着 r1 的最新写入者，保证了 RAW 依赖检测的正确性。

3) I used AI tools to assist with my coding tasks, such as using Copilot to identify and correct coding errors, generate explanatory comments within the code, and utilizing Claude to help design and implement functional modules effectively.

According to 6.3 Academic Integrity: Proper attribution, about the idea of “iterative priority computation (reverse topological order)” in scheduleFast() part, this idea is provided by LLM, and I get the knowledge then revise the original code.

The following is an example of a complete dialogue (about how to revise my errors in the code):

对于这个问题，相关代码我运行后发现出现了运行时间过长的情况，我初查暂时未发现大问题

下面是我的代码：

```
#include "Scheduler.h"

#include <algorithm>
#include <iomanip>
#include <iostream>
#include <limits>
// ...
Show more
```

识别出条件判断逻辑错误导致无限循环。 >

问题一眼就能看到，在 `schedule()` 函数里：

```
cpp

if (ready.empty()) { // ← 逻辑反了！
    int best = selectBestNode(ready, nodes, instructions, policy);
    ...
}
```

应该是 `!ready.empty()`：

```
cpp

if (!ready.empty()) {
```

你写成了 `ready.empty()`，意思变成了“当没有准备好的节点时才调度”，结果就是永远不会调度任何指令，`result.order.size()` 始终为 0，`while` 循环永远不会退出，程序就卡死了。

另外还有一个小问题在 `computeCriticalPath()` 里：

```
cpp

nodes[nodeIdx].priority += maxPath; // ← 用了 +=
```

`priority` 初始值是 0，这里 `+=` 和 `=` 效果一样，所以目前不会出错。但语义上应该写成 `=` 更清晰准确，避免如果 `priority` 被其他地方初始化为非零值时出 bug：

```
cpp

nodes[nodeIdx].priority = maxPath;
```